



RELAZIONE GENE PREDICTION

A cura di:

Andreea Roxana Manolache – mat.245906

Mariaida Perri – mat.239943

Francesco Pio Ruffo – mat.240044

A.A. 2024/25

INTRODUZIONE

Una ORF (Open Reading Frame) è una sequenza di DNA o RNA che può codificare una proteina. È possibile individuare una ORF all'interno di una sequenza nucleotidica quando si incontra:

- Un codone di start (ATG)
- Un codone di stop (TAA, TAG, TGA).

Un ulteriore criterio di individuazione di una ORF è la lunghezza, che dev'essere generalmente superiore ai 150 nucleotidi.

METODO UTILIZZATO

Affinché una ORF sia codificante, è necessaria un'analisi dei promotori, cioè delle regioni precedenti che si trovano tra 200 e 1000 basi prima della regione tradotta.

A questo scopo, si è fatto utilizzo di metodi *ab initio*, senza effettuare confronti basati sull'omologia con altri geni o proteine note.

È stato dunque effettuato un controllo per individuare all'interno di una ORF i sensori di segnale ed i sensori di contenuto.

SENSORI DI SEGNALE

I sensori di segnale servono a identificare sequenze specifiche associate a elementi funzionali dei geni. I sensori di segnale maggiormente impiegati sono:

- Codoni di start (ATG)
- Codoni di stop (TAA, TAG, TGA)
- Motivi promotori, come il TATA box o l'Iniziatore
- Siti regolatori, come la sequenza di Kozak

Nello specifico, il **TATA box** rappresenta una sequenza di DNA in cui sono presenti molteplici basi A e T e si trova circa 25-35 basi prima del sito di inizio della trascrizione. La sua espressione ha la seguente forma:

TATA (A/T) A (A/T)

L'**iniziatore** si colloca invece nella regione che va da -3 a +5 rispetto al sito di inizio della trascrizione. È rappresentato nella forma:

(C/T) (C/T) A (A/C/G/T) (A/T) (C/T) (C/T)

La **sequenza di Kozak** circonda il codone di start negli eucarioti e consente di distinguere tra ATG funzionali e non funzionali. La sua espressione è:

(GCC) GCC (A/G) CC ATG G

SENSORI DI CONTENUTO

I sensori di contenuto sono strumenti che, analizzando la composizione di regioni genomiche estese, valutano la probabilità che una sequenza faccia parte di un gene in base alle sue caratteristiche statistiche. I sensori di contenuto più noti sono:

- Contenuto GC
- Frequenza di utilizzo dei codoni
- Isole CpG

Il **contenuto GC** indica la frazione di nucleotidi C e G presenti all'interno di una sequenza, si calcola cioè come:

$$\frac{\text{numero di G} + \text{numero di C}}{\text{lunghezza totale della sequenza}}$$

Questo valore varia sia in base alla specie considerata, sia tra regioni codificanti e non codificanti (quindi aiuta nell'identificazione di regioni codificanti).

La **frequenza di utilizzo dei codoni** è una misura della frequenza con cui codoni sinonimi vengono utilizzati per la codifica di un amminoacido in un organismo o gene.

Le **isole CpG** sono regioni localizzate del DNA ricche di nucleotidi CG, consistenti di più di 200 basi, con un contenuto CG > 0,5 e un rapporto CpG osservato/atteso > 0,6 (dove per valore atteso si intende il prodotto tra la frequenza di C e la frequenza di G).

APPROCCIO UTILIZZATO

Una volta controllata la presenza di codone di start, codone di stop e una lunghezza superiore alla soglia (250), per verificare le altre caratteristiche che rendono una ORF codificante, si è fatto utilizzo di altri metodi ausiliari. In tal modo, il codice risulta essere modulare e può facilmente essere modificato e testato.

```
1  raccolta_orf = {}
2
3  for orf in orf_iter(str(seq)):
4      id_orf = orf.span()[0]
5      lunghezza = orf.span()[1] - orf.span()[0]
6      if lunghezza > 250:
7          final_results.loc[len(final_results)] = [orf.span()[0], False, False]
8          raccolta_orf[id_orf] = 0
9
10         if sequenza_Kozak(orf.span()[0]):
11             raccolta_orf[id_orf] += 1
12
13         if iniziatore_tata(orf):
14             raccolta_orf[id_orf] += 1
15
16         if isole_CpG(orf):
17             raccolta_orf[id_orf] += 1
18
19         if cg_content((str)(orf)) > 0.45 and cg_content((str)(orf)) < 0.65:
20             raccolta_orf[id_orf] += 1
21
22         #Aggiorniamo il DataFrame. Se hanno punteggio > 1, supponiamo siano
           codificanti
23         if raccolta_orf[id_orf] > 1:
24             results.loc[len(results)] = [id_orf, True, False, Entropia.
               calcola_entropia(str(orf))]
25
26         #Se si trovano corrispondenze su BLAST viene aggiornato il
           parametro
27         if not costr_dataframe_ris_blast(orf.group()).empty:
28             results.loc[len(results)-1, 'checkBLAST'] = True
```

Il dizionario 'raccolta_orf' memorizza le ORF e i relativi punteggi. Il punteggio di una ORF viene incrementato in seguito alla verifica dei sensori di segnale e dei sensori di contenuto. Sulla base del metodo di punteggio implementato, le ORF che presentano punteggio superiore ad 1 (cioè che soddisfano almeno due sensori) sono state aggiunte ad un Data Frame ausiliario con il parametro 'Coding' uguale a 'True'. Inoltre, per queste ultime è stata verificata la corrispondenza su BLAST, aggiornando di conseguenza anche il parametro 'checkBLAST = True'.

```

23         if raccolta_orf[id_orf] > 1:
24             results.loc[len(results)] = [id_orf, True, False, Entropia.
                calcola_entropia(str(orf))]
25
26             #Se si trovano corrispondenze su BLAST viene aggiornato il
                parametro
27             if not costr_dataframe_ris_blast(orf.group()).empty:
28                 results.loc[len(results)-1, 'checkBLAST'] = True

```

Metodo utilizzato per effettuare la ricerca della corrispondenza su BLAST:

```

1  def ricerca_blast(sequenza):
2      res = []
3      # Ricerca su blast
4      x = NCBIWWW.qblast('blastn', 'nt', sequenza, service='
        megablast')
5
6      # Salviamo la risposta
7      blast_records = NCBIXML.parse(x)
8
9      # Iterazione su risultati + salvataggio info principali
10     for blast_record in blast_records:
11         for alignment in blast_record.alignments:
12             for hsp in alignment.hsps:
13                 res.append({
14                     "Sequenza considerata": seq,
15                     "Nome allineamento": alignment.title,
16                     "Lunghezza allineamento": alignment.
                        length,
17                     "E-value": hsp.expect,
18                     "Score": hsp.score,
19                     "Inizio query": hsp.query_start,
20                     "Fine query": hsp.query_end,
21                 })
22     x.close()
23     return res
24
25     # FUNZIONE PER COSTRUIRE IL DATAFRAME CON I DATI RICAVATI
        DALLA RICERCA SU BLAST
26
27     def costruisci_dataframe_risultati_blast(sequenza):
28         df = pd.DataFrame(ricerca_blast(sequenza))
29         return df

```

Il parametro 'service = Megablast' è stato inserito al fine di ottimizzare la ricerca per le sole regioni altamente simili e rendere i risultati prodotti uguali a quelli ottenuti nella ricerca "manuale".

Program Selection	
Optimize for	☒ Highly similar sequences (megablast) ☐ More dissimilar sequences (discontiguous megablast) ☐ Somewhat similar sequences (blastn) Choose a BLAST algorithm 

Durante la fase di progettazione, si è optato per l'impiego di due Data Frame distinti; il Data Frame **'final_results'** contenente l'insieme completo di tutte le ORF identificate (dopo aver applicato il solo filtro relativo alla lunghezza) e **'results'** dedicato esclusivamente alle ORF codificanti.

Questo approccio consente di avere a disposizione tutte le informazioni più rilevanti per ciascun tipo di ORF codificante.

Le informazioni riguardanti le ORF codificanti vengono aggiornate anche nel Data Frame finale.

```
1 for i in range(len(final_results)):
2     for posizione in results['ORFpos']:
3         if final_results.loc[i]['ORFpos'] == posizione:
4             final_results.loc[i, 'Coding'] = True
5             final_results.loc[i, 'BLAST'] = True
```

ULTERIORI AGGIUNTE

ENTROPIA

Inoltre, sono state effettuate ulteriori analisi sulla sequenza, ricorrendo al concetto di entropia.

L'entropia quantifica il contenuto informativo di una sequenza, ed in questo particolare ambito consente di distinguere tra regioni funzionali e non funzionali del genoma.

Nello specifico, le regioni codificanti presentano valori di entropia moderati, mentre quelle non codificanti presentano un'entropia bassa o alta. Per calcolare l'entropia, bisogna valutare prima il contenuto informativo di un evento: un evento è tanto più informativo quanto più è raro e inatteso.

Il contenuto informativo di un evento si calcola come:

$$I(x) = -\log_2(P(x))$$

dove x è l'evento e $P(x)$ è la probabilità dell'evento.

Nel nostro caso, si è proceduto calcolando per ciascun nucleotide la probabilità e il contenuto informativo, ed infine è stata ricavata l'entropia della ORF tramite la seguente espressione:

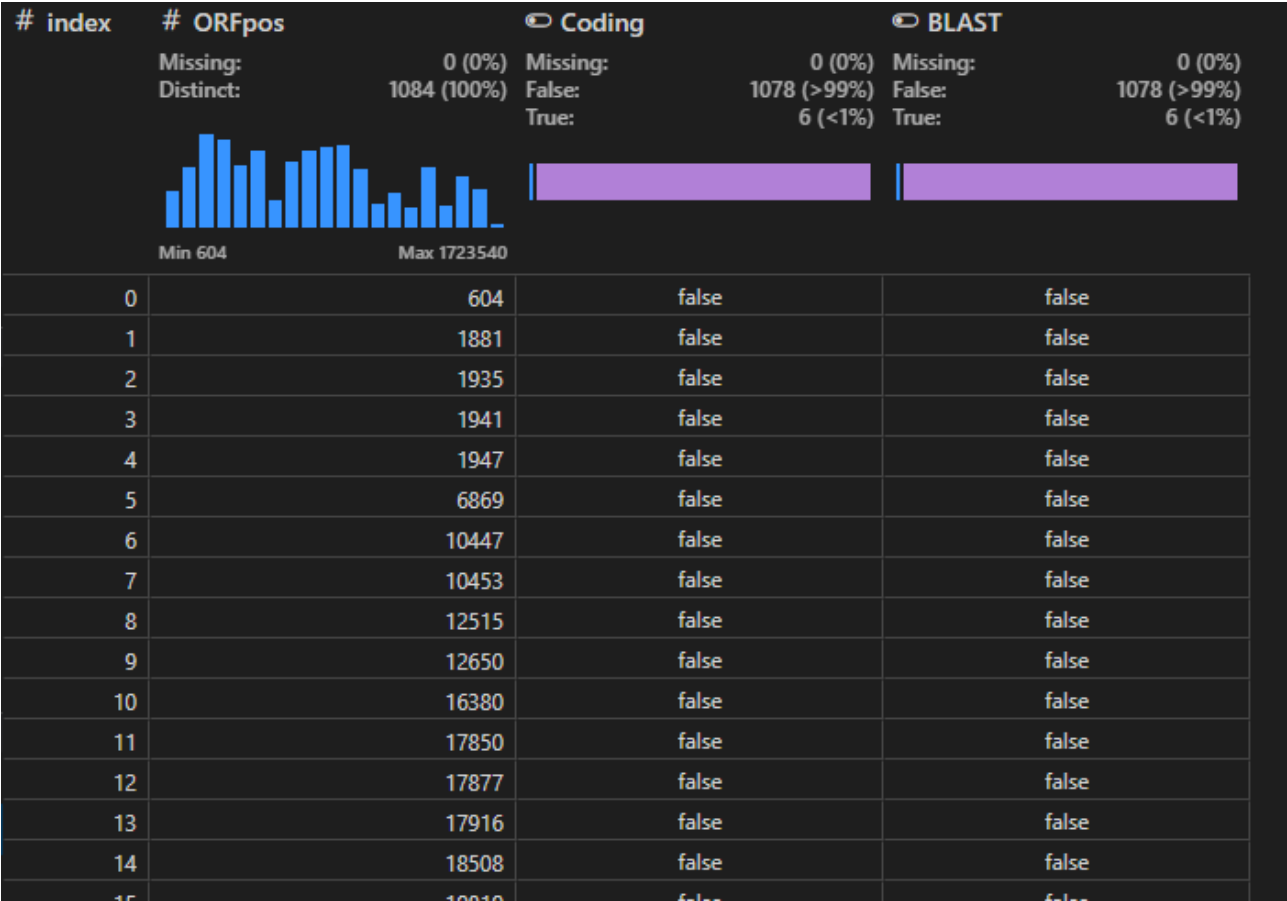
$$H = \sum_x P(x) \cdot I(x) = - \sum_x P(x) \cdot \log_2(P(x))$$

L'entropia consente anche di misurare la similarità tra due sequenze.

```
1 def prob_seq_nucleotidica (seq):
2     letters = ['A','T','C','G']
3     ret = []
4     for i in letters:
5         ret.append (prob_carattere_singolo(i,seq))
6     return ret # ret = [P(A), P(T), P(C), P(G)]
7
8 #Riceve come parametro l'out del metodo prob_seq_nucleotidica
9 def contInformativo (arrayProb):
10     ret = []
11     for elem in arrayProb:
12         ret.append (-math.log(elem,2))
13     return ret # ret = [I(A), I(T), I(C), I (G)]
14
15 def calcola_entropia (sequenza):
16     arrayProb = prob_seq_nucleotidica(sequenza)
17     arrayContInfo = contInformativo (arrayProb)
18     res = 0
19     for i in range (len (arrayProb)):
20         res += arrayProb[i]*arrayContInfo[i]
21     out = str(res)
22     return out
```

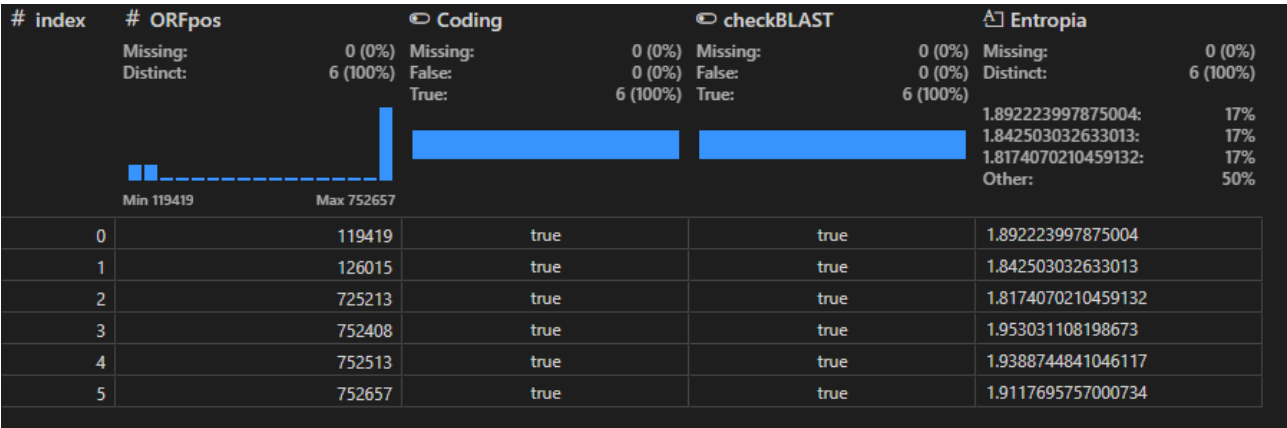
VISUALIZZAZIONE E ANALISI DEI RISULTATI

Per poter analizzare efficacemente i risultati prodotti, è stato fatto utilizzo dell'estensione “Data Wrangler” per Pandas. Tale estensione ha facilitato la visualizzazione dei dati ed ha consentito la rappresentazione dei seguenti report riassuntivi:



**l'immagine precedente rappresenta solo una parte del risultato complessivo.*

Data Frame contenente le ORF codificanti con i rispettivi valori di entropia:

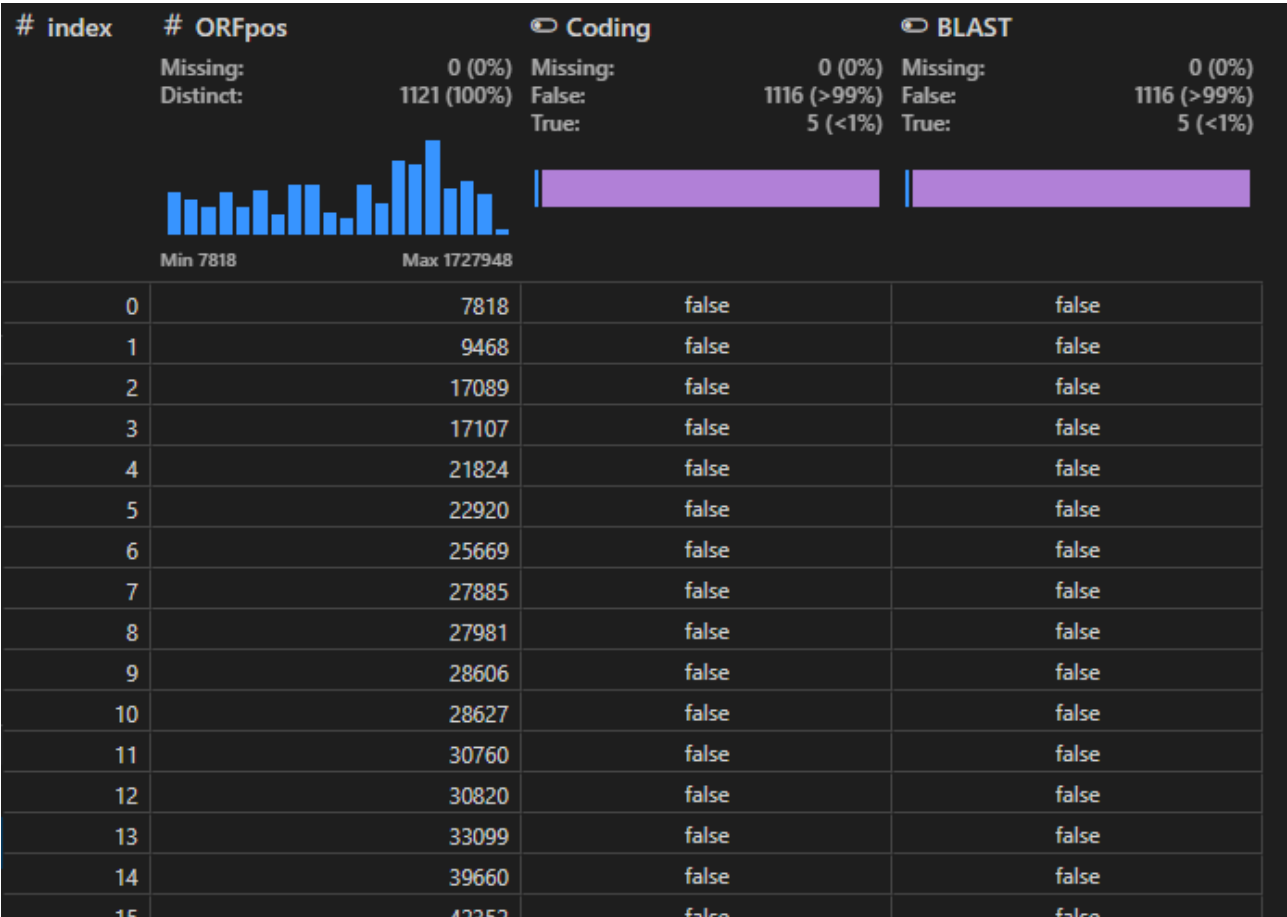


Dai risultati si nota che le sequenze codificanti presentano dei valori di entropia simili.

SEQUENZA INVERSA

Partendo dal codice scritto per analizzare la sequenza primaria, si è ottenuta la sequenza complementare inversa nel seguente modo:

```
1 sequenza_diretta = next(Bio.SeqIO.parse("group_1_seq.fasta",
2   "fasta")).seq
3 seq = sequenza_diretta.reverse_complement()
```



*l'immagine precedente rappresenta solo una parte del risultato complessivo.

Data Frame con le ORF codificanti:

